

```

// api/get-transactions.js
// Fetches balances + 60/90-day transactions for a given entity.
// An entity may have MULTIPLE connected banks (e.g. SOJ = Chase + MACU);
// this aggregates every active plaid_item for the entity and tags each
// account with the bank (institution) it came from.

const { PlaidApi, PlaidEnvironments, Configuration } = require('plaid');
const { createClient } = require('@supabase/supabase-js');

const plaidConfig = new Configuration({
  basePath: PlaidEnvironments[process.env.PLAID_ENV || 'sandbox'],
  baseOptions: {
    headers: {
      'PLAID-CLIENT-ID': process.env.PLAID_CLIENT_ID,
      'PLAID-SECRET': process.env.PLAID_SECRET,
    },
  },
});

const plaidClient = new PlaidApi(plaidConfig);
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_KEY
);

module.exports = async (req, res) => {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'POST, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization');

  if (req.method === 'OPTIONS') return res.status(200).end();
  if (req.method !== 'POST') return res.status(405).json({ error: 'Method not all

  try {
    const { entity_name, days = 90 } = req.body;

    // — Authorize the caller: confirm WHO is asking and that they may see this
    const BRAND_NAMES = { 'The Stick of Joseph': 'SOJ', 'Plain & Precious Publish
    const token = (req.headers.authorization || '').replace(/^Bearer\s+/i, '');
    if (!token) return res.status(401).json({ error: 'Not signed in' });
    const { data: userData, error: authErr } = await supabase.auth.getUser(token)
    if (authErr || !userData || !userData.user) return res.status(401).json({ err
    const { data: prof } = await supabase
      .from('profiles').select('role,brands').eq('email', userData.user.email).ma

```

```

const isAdmin = !!prof && prof.role === 'admin';
const wantKey = BRAND_NAMES[entity_name];
const allowed = isAdmin || (!!prof && Array.isArray(prof.brands) && prof.brands.indexOf(entity_name) !== -1);
if (!allowed) return res.status(403).json({ error: 'Not authorized for this entity' });

// Get ALL active connections for this entity (could be more than one bank)
const { data: items, error: dbError } = await supabase
  .from('plaid_items')
  .select('access_token, institution_name, item_id')
  .eq('entity_name', entity_name)
  .eq('is_active', true);

if (dbError) return res.status(500).json({ error: dbError.message });
if (!items || !items.length) {
  return res.status(404).json({ error: `No connected bank found for ${entity_name}` });
}

const endDate = new Date().toISOString().split('T')[0];
const startDate = new Date(Date.now() - days * 24 * 60 * 60 * 1000)
  .toISOString().split('T')[0];

const allAccounts = [];
const allTxns = [];
const institutions = [];
let needsRefresh = false;

// Loop every bank connected under this entity and merge the results
for (const item of items) {
  try {
    const [txnRes, balRes] = await Promise.all([
      plaidClient.transactionsGet({
        access_token: item.access_token,
        start_date: startDate,
        end_date: endDate,
        options: { count: 500, offset: 0 },
      }),
      plaidClient.accountsBalanceGet({ access_token: item.access_token }),
    ]);

    institutions.push(item.institution_name);

    balRes.data.accounts.forEach(a => {
      allAccounts.push({
        account_id: a.account_id,
        name: a.name,
        mask: a.mask,
        type: a.type,
      });
    });
  } catch (e) {
    console.error(`Error processing item ${item.item_id}: ${e.message}`);
  }
}

```

```

        subtype: a.subtype,
        balance: a.balances.current,
        available: a.balances.available,
        institution: item.institution_name,    // tag each account with its ba
    });
});

txnRes.data.transactions.forEach(t => {
    allTxns.push({
        id: t.transaction_id,
        date: t.date,
        name: t.merchant_name || t.name,
        amount: t.amount,
        category: t.personal_finance_category?.primary || t.category?.[0] ||
        account_id: t.account_id,
        pending: t.pending,
    });
});
} catch (itemErr) {
    // One bank failing shouldn't break the others
    if (itemErr.response?.data?.error_code === 'ITEM_LOGIN_REQUIRED') needsRe
    console.error(`get-transactions item error (${item.institution_name}):`,
        itemErr.response?.data || itemErr.message);
}
}

// Touch last_synced for this entity's items
await supabase.from('plaid_items')
    .update({ last_synced: new Date().toISOString() })
    .eq('entity_name', entity_name);

if (!allAccounts.length && needsRefresh) {
    return res.status(401).json({ error: 'Bank connection needs to be refreshed'
}

const totalBalance = allAccounts.reduce((s, a) => s + (a.balance || 0), 0);

res.status(200).json({
    entity_name,
    institution: [...new Set(institutions)].join(', '),
    needs_refresh: needsRefresh,
    bank_count: items.length,
    accounts: allAccounts,
    transactions: allTxns,
    summary: { total_balance: totalBalance },
});

```

```
    } catch (error) {  
      console.error('Plaid get-transactions error:', error.response?.data || error.  
        res.status(500).json({ error: 'Failed to fetch transactions', details: error.  
      })  
    }  
  };  
};
```